National University
of computer and emerging sciences

Course | Fundamentals of Malware Analysis

Instructor | Mr. Jawad Hassan Nisar

By: Sarmad Farooq

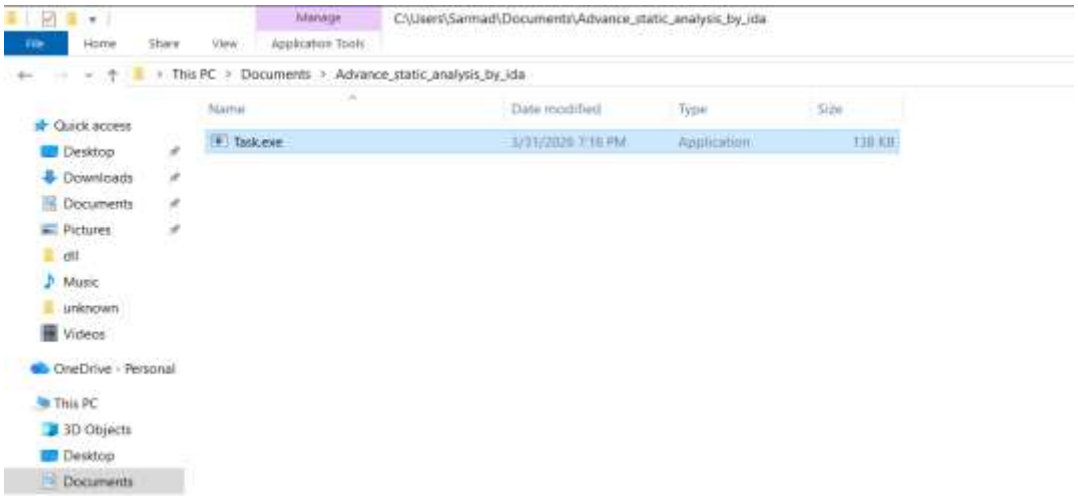
25i-7722

Table of Contents

Steps:

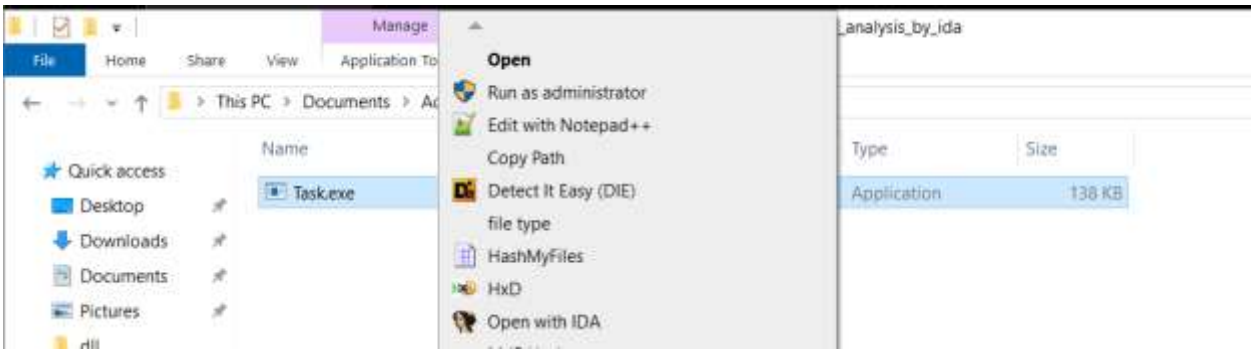
Step1:

Download the provided file and paste in a separate folder.



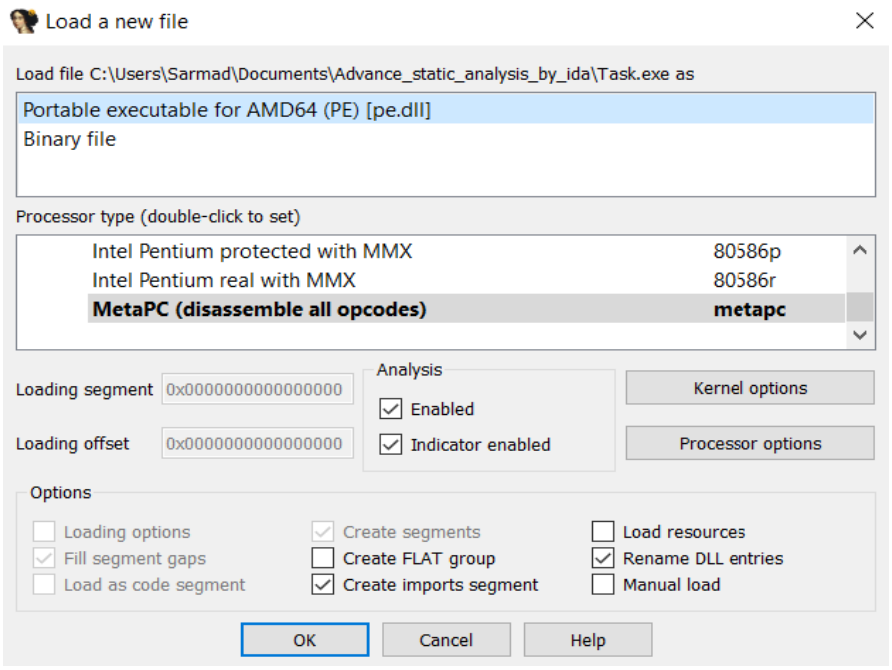
Step2:

By selecting the file right click and select open with IDA.

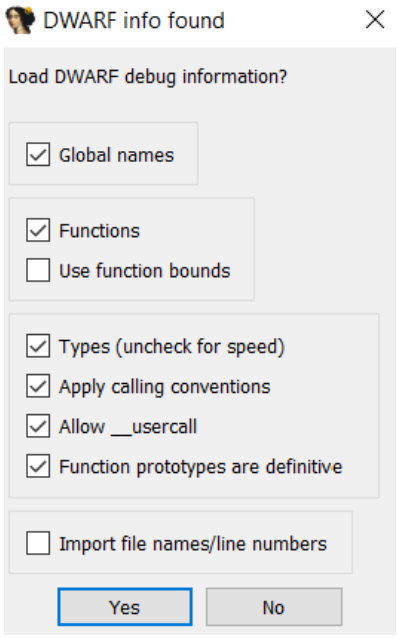


Step3:

Select the Portable executable option and click OK.

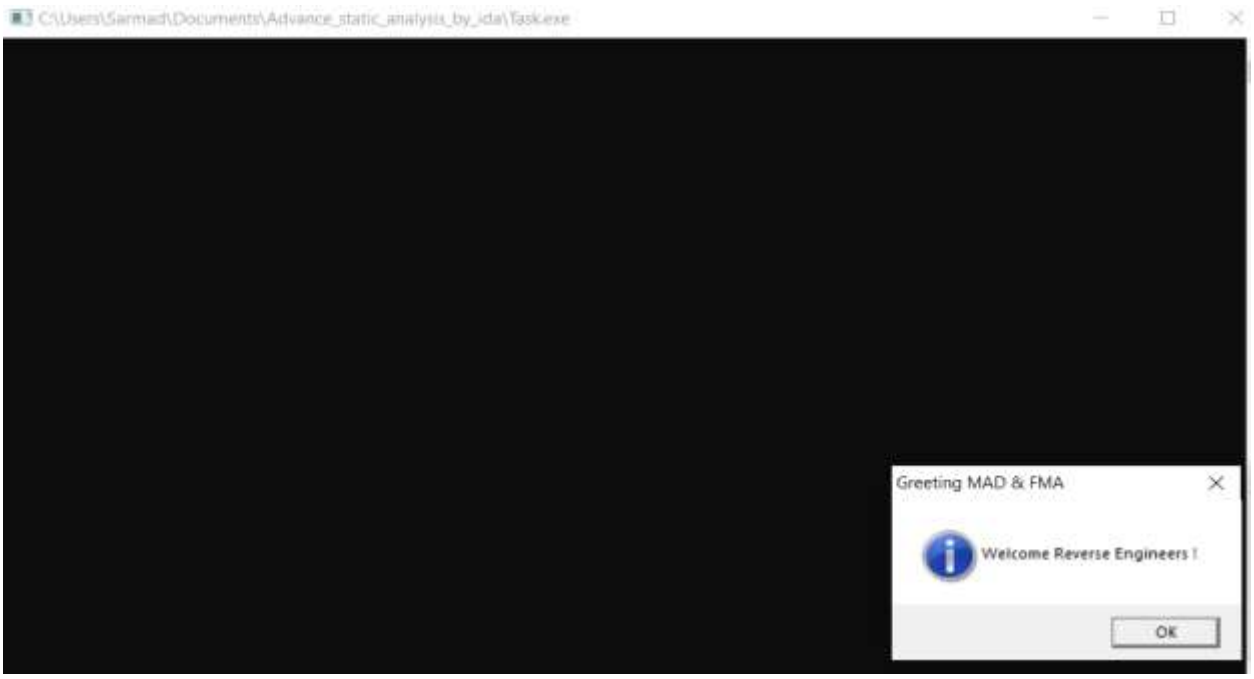


Click Yes.

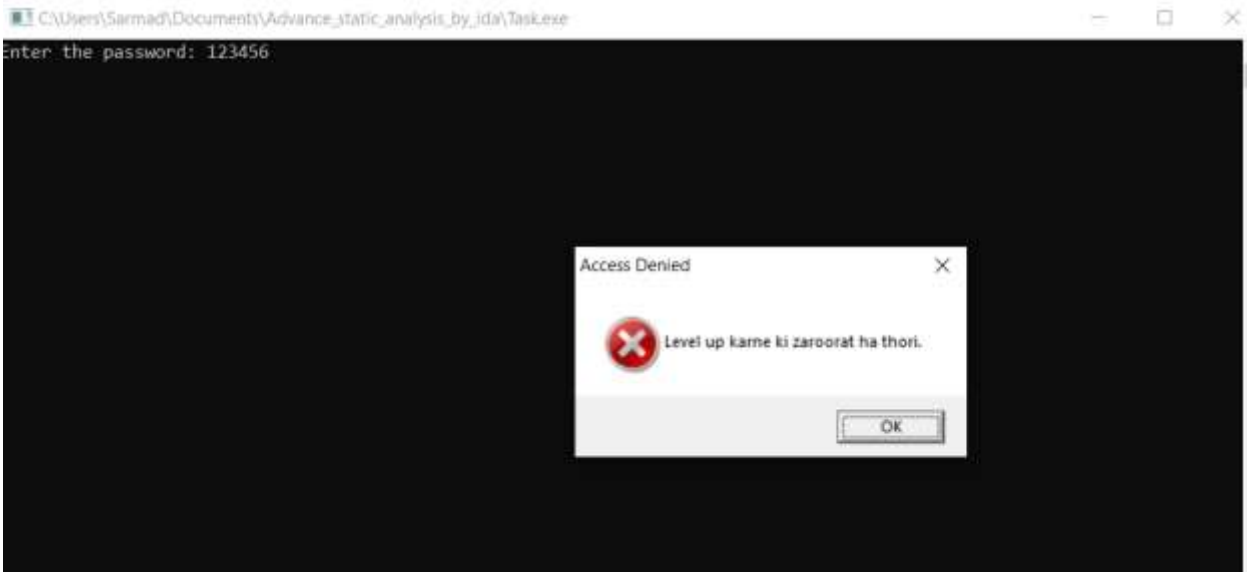


Step4:

Double click the exe. A popup would open click OK and enter any password.

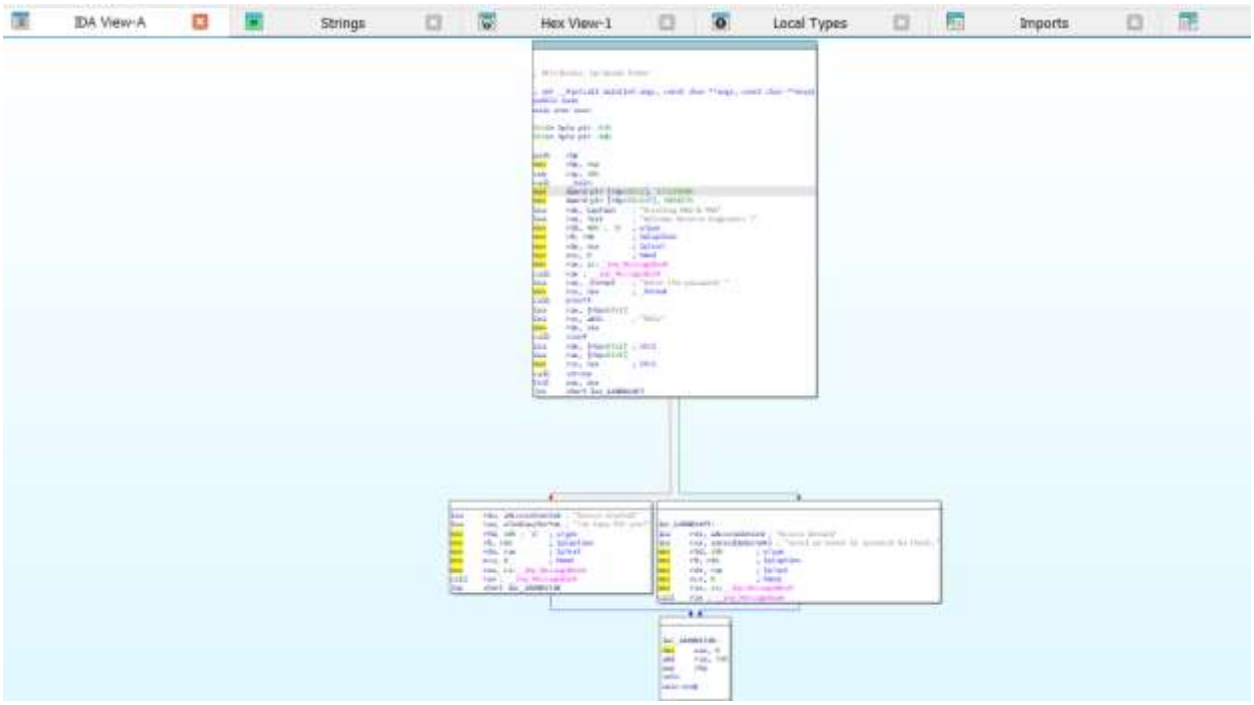


As we can see the password is incorrect and we got an access denied popup.

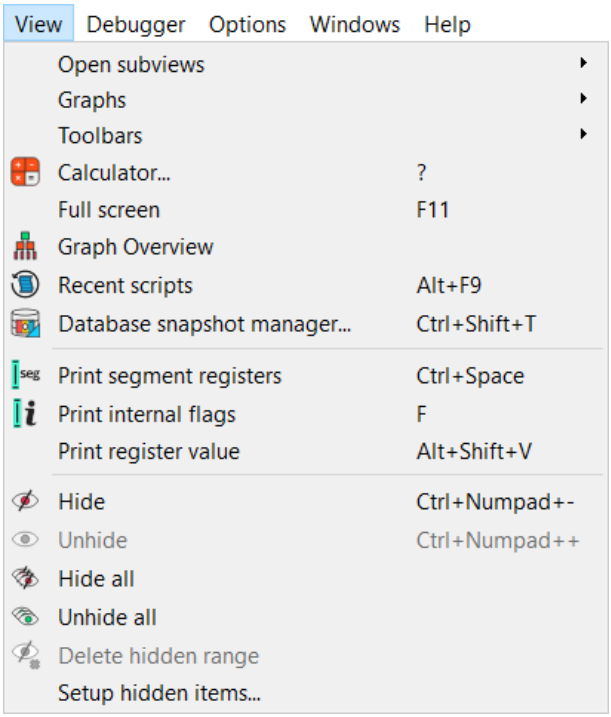


Step5:

You would now see the main IDA graph View.

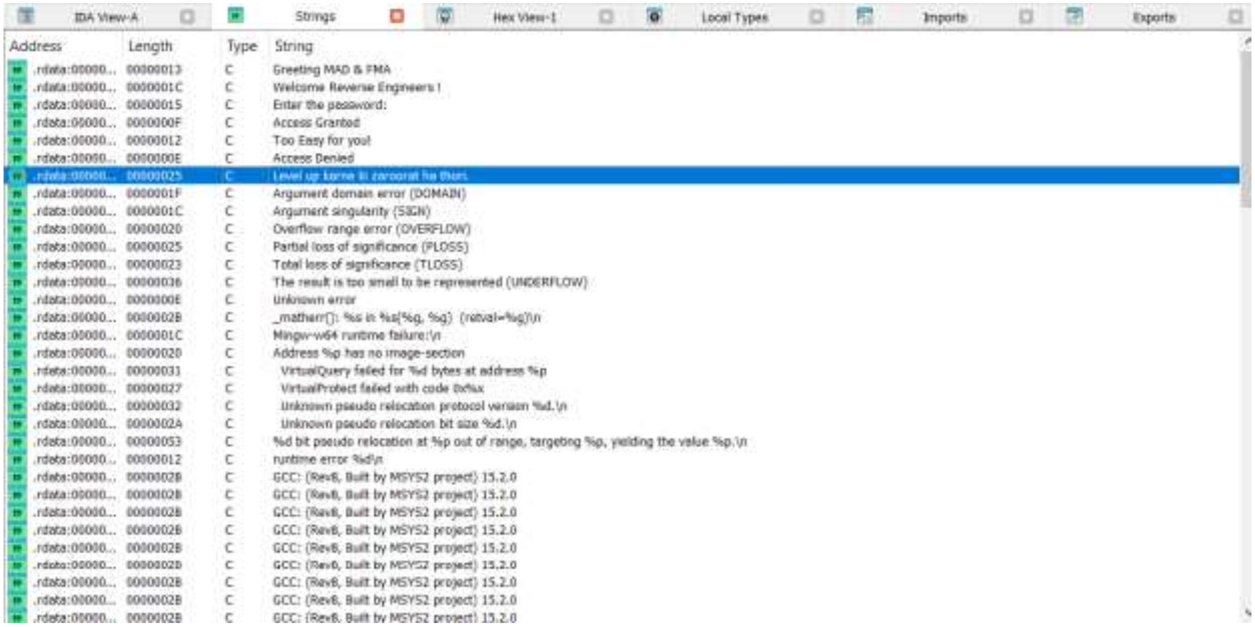


Click on View in the top left corner, then in sub views click on strings.

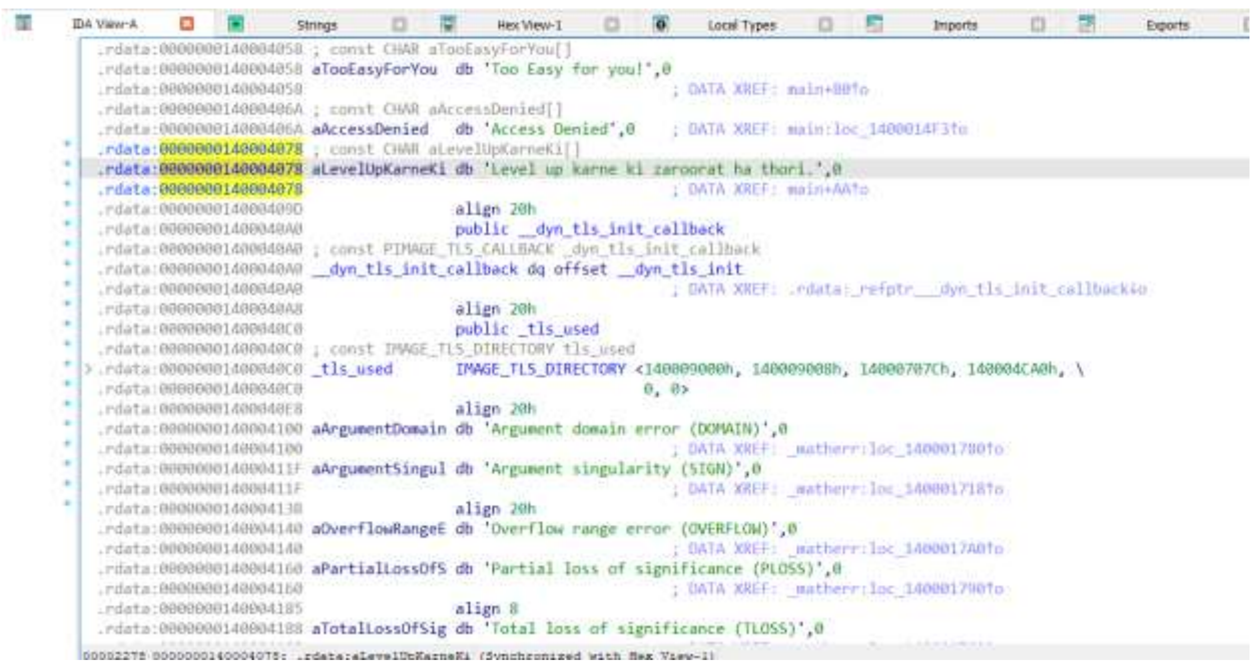


Step6:

Strings view would open. Now here you must analyze the strings.

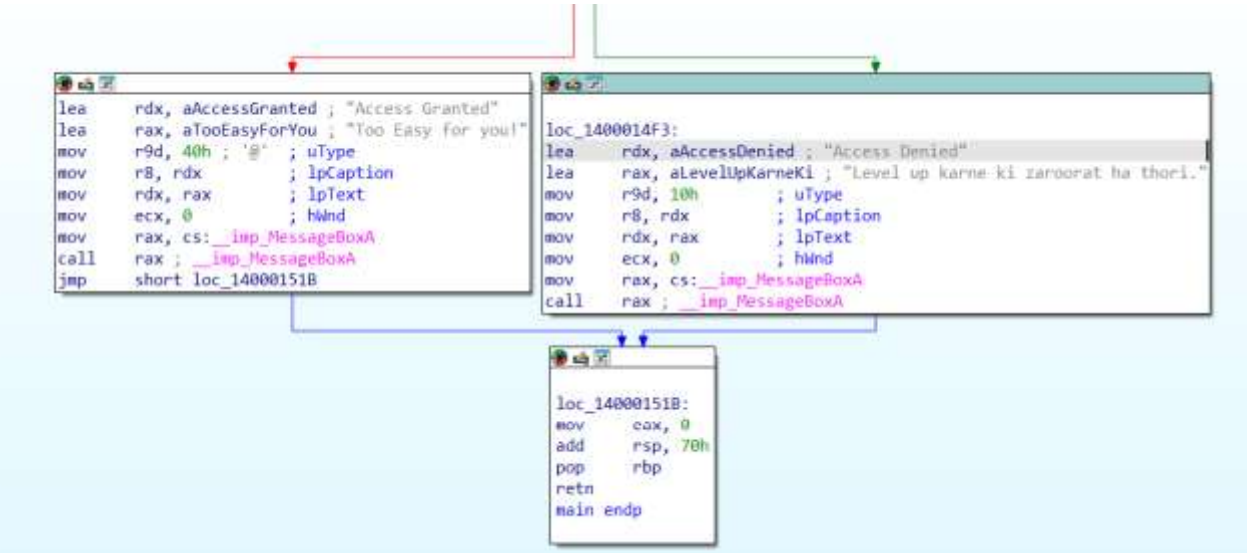


We see Access Denied as of we got the popup earlier. Double click on it. Now its rdata section would open. We can also see the option for its cross-reference location. Double click on the XREF.



Step7:

Now we can clearly see the program flow.



Move to the above block in the graph view which is the programs entry point. Here deep analysis is required. We will analyze the assembly instructions step by step.

See that this instruction is loading address of string "Enter the password: ".

```
lea     rax, _Format    ; "Enter the password: "
mov     rcx, rax        ; _Format
call    printf
lea     rax, [rbp+Str1]
lea     rcx, a49s       ; "%49s"
```

Here its taking user input using **scanf** and loading it in **rax**.

```
mov     rdx, rax
call    scanf
lea     rdx, [rbp+Str2] ; Str2
lea     rax, [rbp+Str1]
mov     rcx, rax        ; Str1
```

By analyzing we came to know that the program is comparing user string Str1 with another string Str2 which could be the original password for this program. We will analyze all of the Str2 instances.

```
; int __fastcall main(int argc, const char **argv, const char **envp)
public main
main proc near

Str2= byte ptr -47h
Str1= byte ptr -40h

push    rbp
mov     rbp, rsp
sub     rsp, 70h
call    __main
mov     dword ptr [rbp+Str2], 37723958h
mov     dword ptr [rbp+Str2+3], 505437h
lea     rdx, Caption    ; "Greeting MAD & FMA"
lea     rax, Text       ; "Welcome Reverse Engineers !"
mov     r9d, 40h ; '@'  ; uType
mov     r8, rdx         ; lpCaption
mov     rdx, rax        ; lpText
mov     ecx, 0          ; hWnd
mov     rax, cs:__imp_MessageBoxA
call    rax ; __imp_MessageBoxA
lea     rax, _Format    ; "Enter the password: "
mov     rcx, rax        ; _Format
call    printf
lea     rax, [rbp+Str1]
lea     rcx, a49s       ; "%49s"
mov     rdx, rax
call    scanf
lea     rdx, [rbp+Str2] ; Str2
lea     rax, [rbp+Str1]
```

We can see that here program stores 4 bytes into **Str2** (**37 72 39 58**). Must note that **little endian** would be followed so in reverse (**58 39 72 37**).

```
mov     dword ptr [rbp+Str2], 37723958h
mov     dword ptr [rbp+Str2+3], 505437h
```

This instruction is storing more bytes starting at offset **+3** (**37 54 50**).

```
mov     dword ptr [rbp+Str2], 37723958h
mov     dword ptr [rbp+Str2+3], 505437h
```

Step8:

So now we have **58 39 72 37** at indexes 0 1 2 3 respectively and then more **37 54 50**, that are staring after index 3 meaning overwriting the original 3rd index. So, finally forming **58 39 72 37 54 50**.

Now use the **ASCII table** **https://michaelgoerz.net/refcards/ascii_a4.pdf** to find the Characters.

REGULAR ASCII CHART (character codes 0 – 127)

000d	00h		(nul)	016d	10h	►	(dle)	032d	20h			048d	30h	0		064d	40h	@		080d	50h	P		096d	60h	`		112d	70h	p	
001d	01h	☺	(soh)	017d	11h	◄	(dc1)	033d	21h	!		049d	31h	1		065d	41h	A		081d	51h	Q		097d	61h	a		113d	71h	q	
002d	02h	☻	(stx)	018d	12h	↕	(dc2)	034d	22h	"		050d	32h	2		066d	42h	B		082d	52h	R		098d	62h	b		114d	72h	r	
003d	03h	♥	(etx)	019d	13h	‼	(dc3)	035d	23h	#		051d	33h	3		067d	43h	C		083d	53h	S		099d	63h	c		115d	73h	s	
004d	04h	♦	(eot)	020d	14h	¶	(dc4)	036d	24h	$		052d	34h	4		068d	44h	D		084d	54h	T		100d	64h	d		116d	74h	t	
005d	05h	♣	(enq)	021d	15h	§	(nak)	037d	25h	%		053d	35h	5		069d	45h	E		085d	55h	U		101d	65h	e		117d	75h	u	
006d	06h	♠	(ack)	022d	16h	▬	(syn)	038d	26h	&		054d	36h	6		070d	46h	F		086d	56h	V		102d	66h	f		118d	76h	v	
007d	07h	•	(bel)	023d	17h	↨	(etb)	039d	27h	'		055d	37h	7		071d	47h	G		087d	57h	W		103d	67h	g		119d	77h	w	
008d	08h	◘	(bs)	024d	18h	↑	(can)	040d	28h	(		056d	38h	8		072d	48h	H		088d	58h	X		104d	68h	h		120d	78h	x	
009d	09h		(tab)	025d	19h	↓	(em)	041d	29h	)		057d	39h	9		073d	49h	I		089d	59h	Y		105d	69h	i		121d	79h	y	
010d	0Ah	■	(lf)	026d	1Ah		(eof)	042d	2Ah	*		058d	3Ah	:		074d	4Ah	J		090d	5Ah	Z		106d	6Ah	j		122d	7Ah	z	
011d	0Bh	♂	(vt)	027d	1Bh	←	(esc)	043d	2Bh	+		059d	3Bh	;		075d	4Bh	K		091d	5Bh	[		107d	6Bh	k		123d	7Bh	{	
012d	0Ch		(np)	028d	1Ch	∟	(fs)	044d	2Ch	,		060d	3Ch	<		076d	4Ch	L		092d	5Ch	\		108d	6Ch	l		124d	7Ch	|	
013d	0Dh	♪	(cr)	029d	1Dh	↔	(gs)	045d	2Dh	-		061d	3Dh	=		077d	4Dh	M		093d	5Dh	]		109d	6Dh	m		125d	7Dh	}	
014d	0Eh	♫	(so)	030d	1Eh	▲	(rs)	046d	2Eh	.		062d	3Eh	>		078d	4Eh	N		094d	5Eh	^		110d	6Eh	n		126d	7Eh	~	
015d	0Fh	☼	(si)	031d	1Fh	▼	(us)	047d	2Fh	/		063d	3Fh	?		079d	4Fh	O		095d	5Fh	_		111d	6Fh	o		127d	7Fh	⌂	

EXTENDED ASCII CHART (character codes 128 – 255) LATIN1/CP1252

128d	80h	€		144d	90h			160d	A0h			176d	B0h	°		192d	C0h	À		208d	D0h	Ð		224d	E0h	à		240d	F0h	ð	
129d	81h			145d	91h	'		161d	A1h	¡		177d	B1h	±		193d	C1h	Á		209d	D1h	Ñ		225d	E1h	á		241d	F1h	ñ	
130d	82h	‚		146d	92h	'		162d	A2h	¢		178d	B2h	²		194d	C2h	Â		210d	D2h	Ò		226d	E2h	â		242d	F2h	ò	
131d	83h	ƒ		147d	93h	"		163d	A3h	£		179d	B3h	³		195d	C3h	Ã		211d	D3h	Ó		227d	E3h	ã		243d	F3h	ó	
132d	84h	„		148d	94h	"		164d	A4h	¤		180d	B4h	´		196d	C4h	Ä		212d	D4h	Ô		228d	E4h	ä		244d	F4h	ô	
133d	85h	…		149d	95h	•		165d	A5h	¥		181d	B5h	µ		197d	C5h	Å		213d	D5h	Õ		229d	E5h	å		245d	F5h	õ	
134d	86h	†		150d	96h	–		166d	A6h	¦		182d	B6h	¶		198d	C6h	Æ		214d	D6h	Ö		230d	E6h	æ		246d	F6h	ö	
135d	87h	‡		151d	97h	—		167d	A7h	§		183d	B7h	·		199d	C7h	Ç		215d	D7h	×		231d	E7h	ç		247d	F7h	÷	
136d	88h	ˆ		152d	98h	˜		168d	A8h	¨		184d	B8h	¸		200d	C8h	È		216d	D8h	Ø		232d	E8h	è		248d	F8h	ø	
137d	89h	‰		153d	99h	™		169d	A9h	©		185d	B9h	¹		201d	C9h	É		217d	D9h	Ù		233d	E9h	é		249d	F9h	ù	
138d	8Ah	Š		154d	9Ah	š		170d	AAh	ª		186d	BAh	º		202d	CAh	Ê		218d	DAh	Ú		234d	EAh	ê		250d	FAh	ú	
139d	8Bh	‹		155d	9Bh	›		171d	ABh	«		187d	BBh	»		203d	CBh	Ë		219d	DBh	Û		235d	EBh	ë		251d	FBh	û	
140d	8Ch	Œ		156d	9Ch	œ		172d	ACh	¬		188d	BCh	¼		204d	CCh	Ì		220d	DCh	Ü		236d	ECh	ì		252d	FCh	ü	
141d	8Dh			157d	9Dh			173d	ADh			189d	BDh	½		205d	CDh	Í		221d	DDh	Ý		237d	EDh	í		253d	FDh	ý	
142d	8Eh	Ž		158d	9Eh	ž		174d	AEh	®		190d	BEh	¾		206d	CEh	Î		222d	DEh	Þ		238d	EEh	î		254d	FEh	þ	
143d	8Fh			159d	9Fh	Ÿ		175d	AFh	¯		191d	BFh	¿		207d	CFh	Ï		223d	DFh	ß		239d	EFh	ï		255d	FFh	ÿ	

Hexadecimal to Binary

0	0000	4	0100	8	1000	C	1100
1	0001	5	0101	9	1001	D	1101
2	0010	6	0110	A	1010	E	1110
3	0011	7	0111	B	1011	F	1111

Groups of ASCII-Code in Binary

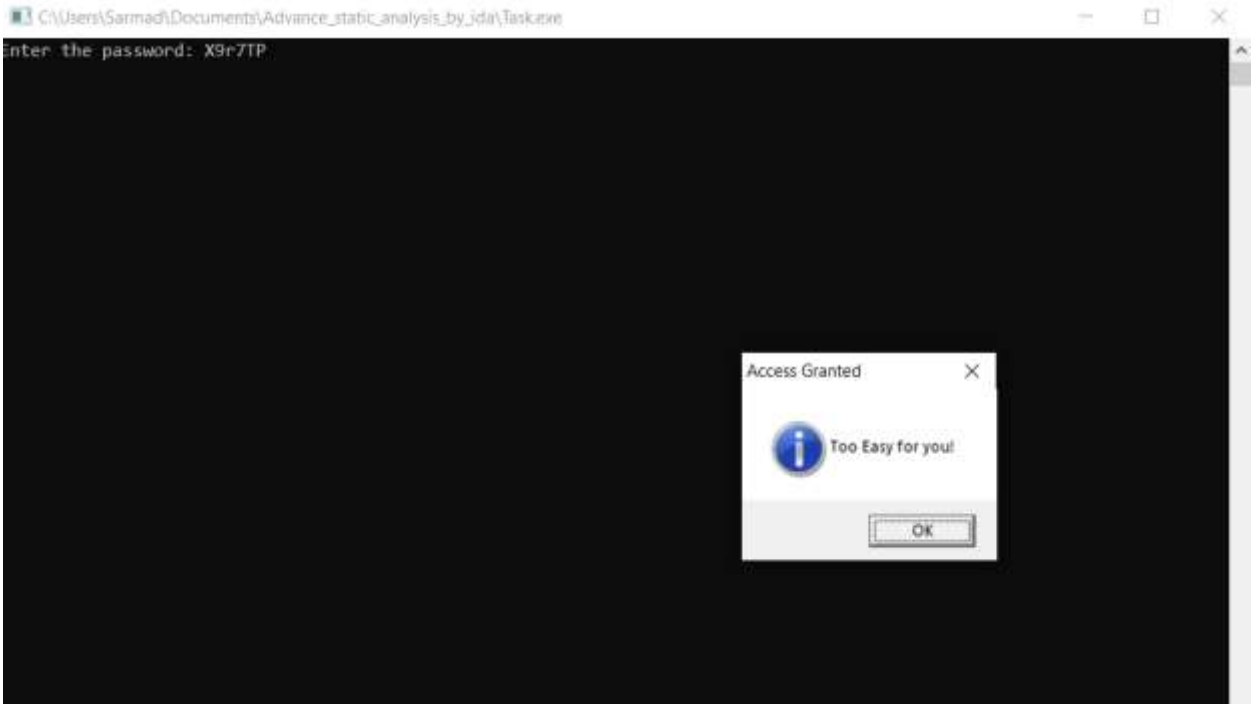
Bit 6	Bit 5	Group
0	0	Control Characters
0	1	Digits and Punctuation
1	0	Upper Case and Special
1	1	Lower Case and Special

Final finding: **X 9 r 7 T P**

Step9:

Double click on the exe again. Enter the found password.

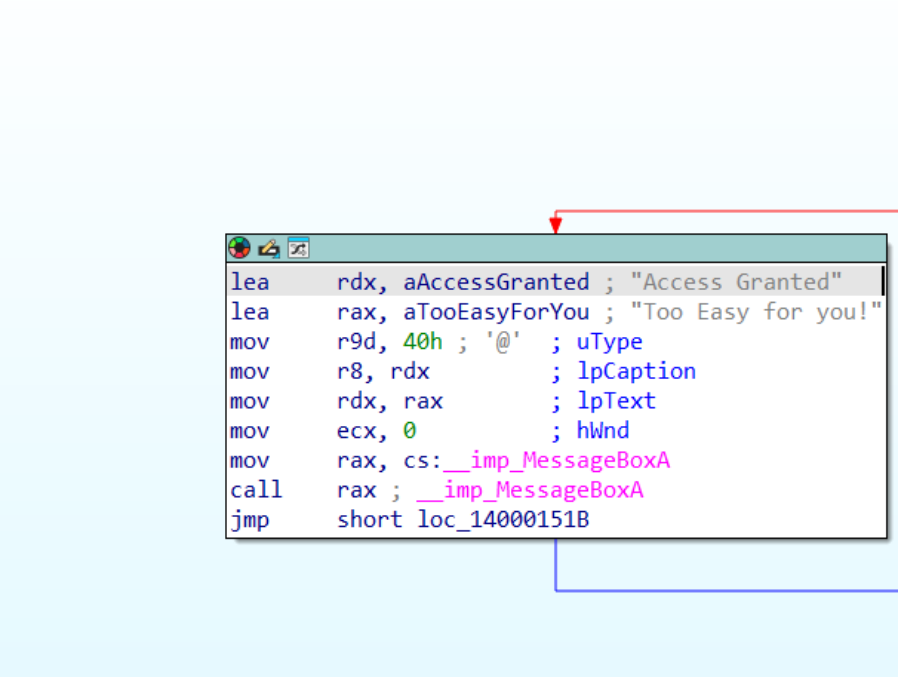
Hurray! You cracked the password by static analysis.



Now you can see in IDR String section

```
.rdata:0000000140004044 a49s            db '%49s',0             ; DATA XREF: main+56↑o
.rdata:0000000140004049 ; const CHAR aAccessGranted[]
.rdata:0000000140004049 aAccessGranted  db 'Access Granted',0   ; DATA XREF: main+79↑o
.rdata:0000000140004058 ; const CHAR aTooEasyForYou[]
.rdata:0000000140004058 aTooEasyForYou  db 'Too Easy for you!',0
.rdata:0000000140004058                                         ; DATA XREF: main+80↑o
.rdata:000000014000406A ; const CHAR aAccessDenied[]
.rdata:000000014000406A aAccessDenied   db 'Access Denied',0    ; DATA XREF: main:loc_1400014F3↑o
.rdata:0000000140004078 ; const CHAR aLevelUpKarneKi[]
```

When I click on XRFF it Redirect to Access Granted Section of Graph



Task Completed